POSSIBLE METHODOLOGY WE MAY FOLLOW

clinical problem → data → preprocessing → major concepts → Module 1 → Module 2 → patient-level aggregation → final triage → research gaps → evaluation.

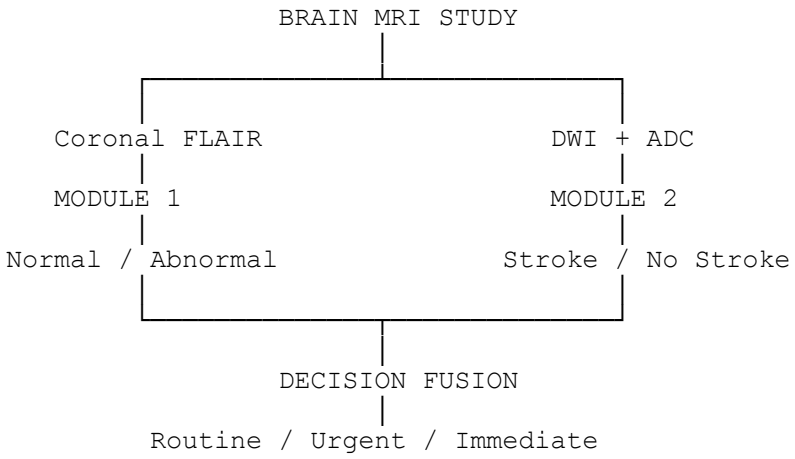
That way they understand *why* each technical choice exists instead of memorizing an architecture.

1. What are we actually building?

Our project is a **brain MRI triage system**.

The aim is **not to diagnose every disease or replace the radiologist**. The aim is to look at an MRI examination before it reaches the radiologist and decide how urgently it should be reviewed.

We have two separate AI modules:



Broadly:

Module 1

- input: coronal T2-FLAIR
- task: general abnormality detection
- output: Normal / Abnormal

Module 2

- input: paired DWI + ADC
- task: acute ischemic stroke detection
- output: Stroke / No Stroke

Then the two results are combined for final triage.

---

2. Why do we need two modules?

FLAIR and DWI/ADC answer different clinical questions.

FLAIR

FLAIR is useful for detecting many general abnormalities such as:

- white-matter abnormalities
- edema
- tumors
- infarcts
- demyelination
- structural abnormalities
- some infections/inflammatory changes

So it is useful for broad screening:

"Does this brain MRI look generally normal or abnormal?"

DWI + ADC

These are particularly important for **acute ischemic stroke**.

A typical acute infarct may show:

```
DWI  → high signal
ADC  → corresponding low signal
```

So the stroke module asks:

"Is there evidence of an acute ischemic stroke?"

That is why stroke gets its own dedicated module instead of being treated as just another FLAIR abnormality.

---

3. The main concepts everyone in your team should understand

Before architecture, they should know these concepts:

```
MRI sequences
DICOM
MRI slices and volumes
DINOv2
Vision Transformer (ViT)
Feature embeddings
Transfer learning
Frozen vs fine-tuned backbone
Patient-level vs slice-level classification
Feature fusion
Pooling
Attention pooling
Position-aware attention
Patient-level train/validation/test splitting
Explainability
Decision fusion
```

I'll explain each as we go.

---

4. What is one patient's MRI data actually like?

MRI is not usually just one picture.

A sequence contains **multiple 2D slices**.

For example:

```
Patient A - Coronal FLAIR
    Slice 01
    Slice 02
    Slice 03
    ...
    Slice 25
```

Another patient may have:

```
Patient B
    Slice 01
    ...
    Slice 21
```

That is completely normal.

Different patients can have different numbers of slices.

A DICOM series may therefore look like:

```
Patient_001/
    FLAIR/
        IMG0001.dcm
```

```
IMG0002.dcm
IMG0003.dcm
...
IMG0025.dcm
```

Each `.dcm` is usually one slice, while the complete series represents the MRI sequence.

---

5. What are we doing with the hospital DICOM data?

Right now, you and your dad are collecting the original hospital **DICOM studies**.

That is the correct thing to preserve as your source data.

The rough workflow later should be:

```
Hospital DICOM
      |
      ▼
De-identification
(RSNA DICOM Anonymizer)
      |
      ▼
Metadata QC
(pydicom)
      |
      ▼
Visual QC
(RadiAnt / OsiriX etc.)
      |
      ▼
Sequence identification
FLAIR / DWI / ADC
      |
      ▼
Research dataset
```

Important:

Keep the **de-identified DICOM copy as the master research source**.

Do not throw away the original medical-image information and keep only screenshots/JPGs.

---

6. What does de-identification mean?

DICOM contains two kinds of information:

Pixel data

The actual MRI image.

Metadata

Things like:

```
Patient Name
Patient ID
Date of Birth
Study Date
Series Description
Scanner manufacturer
Slice location
Pixel spacing
etc.
```

You remove identifying information such as:

```
Name
hospital ID
DOB
accession number
personal identifiers
```

while preserving research-useful imaging metadata.

After RSNA anonymization, we should verify the result with **pydicom**.

For example, pydicom can check:

```
PatientName
PatientID
StudyInstanceUID
SeriesInstanceUID
SeriesDescription
Modality
Rows
Columns
PixelSpacing
SliceThickness
```

This is **metadata QC**, not clinical diagnosis.

---

7. Module 1 dataset

Module 1 is:

Coronal FLAIR → Normal / Abnormal

A possible clean directory after de-identification could look like:

```
Module1_FLAIR/
│
├── Normal/
│   ├── M1_0001/
│   │
│   │       ├── IMG0001.dcm
│   │       ├── IMG0002.dcm
│   │       ├── IMG0003.dcm
│   │       └── ...
│   │
│   ├── M1_0002/
│   │
│   │       └── ...
│   └── ...
│
└── Abnormal/
    ├── M1_0201/
    │
    │       ├── IMG0001.dcm
    │       ├── IMG0002.dcm
    │       └── ...
    │
    └── ...
```

But for research code, I actually prefer not relying only on folder names for labels.

Keep a **CSV metadata file as the ground truth**.

Csv data may have – study id of patient, label, diagnosis, other relevant columns if needed.

---

8. Module 2 dataset

Module 2 needs paired:

DWI + ADC

So one patient could look like:

```
Module2_Stroke/
│
├── M2_0001/
│   ├── DWI/
│   │   ├── IMG0001.dcm
│   │   ├── IMG0002.dcm
│   │   └── ...
│   │
│   └── ADC/
│       ├── IMG0001.dcm
│       ├── IMG0002.dcm
│       └── ...
│
├── M2_0002/
```

```
│   ├── DWI/
│   └── ADC/
│
└── ...
```

Ground-truth CSV:

Contains : Study id of patient, stroke_label, other columns if needed.

Stroke
→ radiologist-confirmed acute ischemic infarction with appropriate DWI/ADC findings.

No Stroke
→ no acute ischemic infarction.

---

9. How do we pair DWI and ADC?

We don't randomly combine slices.

Corresponding anatomical slices should match.

Conceptually:

```
DWI slice 1  ↔ ADC slice 1
DWI slice 2  ↔ ADC slice 2
DWI slice 3  ↔ ADC slice 3
...
```

We check things like:

```
slice order
orientation
number of slices
pixel spacing
image position
series geometry
```

If needed, intra-patient registration/alignment can be performed.

The result should be:

```
(DWI1, ADC1)
(DWI2, ADC2)
(DWI3, ADC3)
...
```

These are the units that Module 2 processes.

---

10. Can Module 1 and Module 2 have different training patients?

Yes.

This is important.

You do **not need** the same patients for training both modules.

For example:

```
MODULE 1 TRAINING
600 FLAIR patients

MODULE 2 TRAINING
600 different DWI+ADC patients
```

That is fine because the networks are solving different tasks.

The only strict condition is:

A patient used for the final integrated test must never have appeared in training either module.

---

11. One correction about validation

For model development, each individual module still needs its **own validation set**.

So a clean development structure is:

```
Module 1 dataset
    Train
    Validation

Module 2 dataset
    Train
    Validation
```

These validation sets are used for things like:

```
hyperparameter tuning
early stopping
threshold selection
frozen vs fine-tuned comparison
```

Then separately you can have a **combined integrated validation/test cohort** containing all sequences.

That combined cohort is used to test the complete system.

---

12. What does the final integrated dataset look like?

For the final system, each patient needs (i.e for testing and validation) :

```
FLAIR
DWI
ADC
```

Example directory:

```
Integrated_Test/
│
├── TEST_001/
│   ├── FLAIR/
│   │   ├── IMG0001.dcm
│   │   └── ...
│   │
│   ├── DWI/
│   │   ├── IMG0001.dcm
│   │   └── ...
│   │
│   └── ADC/
│       ├── IMG0001.dcm
│       └── ...
│
├── TEST_002/
│   ├── FLAIR/
│   ├── DWI/
│   └── ADC/
│
└── ...
```

CSV:

patient_id	flair_path	dwi_path	adc_path	flair_label	stroke_label	expected_priority
TEST_001	...	...	...	Normal	No Stroke	Routine
TEST_002	...	...	...	Abnormal	No Stroke	Urgent
TEST_003	...	...	...	Abnormal	Stroke	Immediate

This cohort lets us evaluate the **whole pipeline**.

---

13. Absolutely critical rule: patient-level splitting

Never split like this:

```
Patient A Slice 1 → Train
Patient A Slice 2 → Train
Patient A Slice 20 → Test   ❌
```

That creates leakage.

MRI slices belonging to the same patient are highly similar.

The model may therefore appear to perform extremely well simply because it has already seen another slice from that patient.

Correct:

```
Patient A → TRAIN ONLY
Patient B → TRAIN ONLY
Patient C → VALIDATION ONLY
Patient D → TEST ONLY
```

All slices follow their patient.

---

14. What is DINOv2 doing?

DINOv2 is a **self-supervised Vision Transformer foundation model**.

It was pretrained on a huge collection of ordinary images without requiring conventional manual class labels.

It learned useful visual patterns such as:

```
shapes
edges
textures
structures
spatial relationships
semantic patterns
```

We reuse those representations for MRI.

Important:

DINOv2 is primarily a **2D image encoder**.

It does not directly take our complete 25-slice MRI study.

Instead:

```
Slice 1 → DINOv2 → Feature vector
Slice 2 → DINOv2 → Feature vector
Slice 3 → DINOv2 → Feature vector
...
```

For DINOv2-Base, one possible global representation is approximately:

768768

features per slice.

---

15. What is a feature vector?

Instead of retaining the entire raw image, DINOv2 converts the image into numbers describing its visual characteristics.

For example:

```
MRI slice
   ↓
DINOv2
   ↓
[0.32, -0.81, 1.12, 0.04, ...]
          768 values
```

You shouldn't interpret one number as:

feature 327 = tumor.

It is a learned high-dimensional representation.

---

16. Frozen DINOv2 vs fine-tuning

Initially we plan:

```
DINOv2 = frozen
```

Meaning its original weights are not changed.

We train only the small new layers that we add.

Why?

Because our hospital dataset may not be huge.

Freezing gives:

- lower training cost
- less overfitting
- faster experiments
- easier debugging

Then an ablation can compare:

```
Frozen DINOv2
vs
Last few blocks fine-tuned
```

We should let validation performance decide whether fine-tuning helps.

---

17. Module 1 architecture

The proposed Module 1 is:

```
Coronal FLAIR DICOM
        |
        ▼
Preprocessing / QC
        |
        ▼
Relevant ordered slices
        |
        ▼
DINOv2 applied to each slice
        |
        ▼
Slice embeddings
        |
        + Relative slice position
        |
        ▼
Position-aware attention pooling
        |
        ▼
One patient embedding
        |
        ▼
MLP classifier
        |
        ▼
Normal / Abnormal
```

---

18. What is slice-level analysis?

Suppose Patient A has 25 slices.

A purely slice-level system would do:

```
Slice 1 → Normal
Slice 2 → Normal
Slice 3 → Abnormal
...
Slice 25 → Normal
```

But then:

What is the prediction for the patient?

That's the problem.

In the real world the radiologist is interested in:

Patient A's examination → Normal or Abnormal

not 25 unrelated predictions.

---

19. What is patient-level analysis?

Patient-level analysis means:

```
ALL relevant slices
        ↓
combined intelligently
        ↓
one representation
        ↓
ONE final patient prediction
```

So:

```
25 slices
   ↓
Patient A
   ↓
Abnormal
```

That is what our attention pooling does.

---

20. Why not simply average all slices?

The simplest method is:

Mean pooling

Suppose DINO generates:

```
z1
z2
z3
...
z25
```

Mean pooling calculates:

zpatient=1N∑iziz_{patient} = \frac{1}{N}\sum_i z_i

This gives **every slice equal importance**.

Problem:

Maybe only 2 out of 25 slices show a small lesion.

Mean pooling treats those 2 slices exactly like the 23 less informative slices.

---

21. What about majority voting?

Another method is:

```
Slice 1 → Normal
Slice 2 → Normal
Slice 3 → Abnormal
...
```

Then count:

```
Normal = 20
Abnormal = 5
```

Patient prediction:

```
Normal
```

This is **majority voting**.

Problem:

What if those 5 abnormal slices contain a genuine lesion?

A minority of slices can still contain the clinically critical information.

Majority voting also throws away the rich DINO feature vectors and retains only:

```
0 / 1
```

---

22. What is max pooling?

Another possibility is to take the strongest activation across slices.

But this can make the decision depend too heavily on one extreme slice and does not learn which patterns are useful across the study.

---

23. Attention pooling

Attention pooling learns:

"Which slices should contribute more strongly to this patient's final prediction?"

Suppose there are five slices.

The network could learn:

Slice	Attention
1	0.03
2	0.07
3	0.60
4	0.25
5	0.05

Then:

zpatient=0.03z1+0.07z2+0.60z3+0.25z4+0.05z5z_{patient} = 0.03z_1 + 0.07z_2 + 0.60z_3 + 0.25z_4 + 0.05z_5

The weights are learned automatically.

So:

```
Mean pooling
→ all slices equally important

Majority voting
→ all slice decisions equally important

Attention pooling
→ importance is learned from the data
```

This is one of the most important concepts in our architecture.

---

24. What exactly is position-aware attention?

Ordinary attention receives:

```
slice features
```

But MRI slices form an **ordered anatomical stack**.

Slice 3 and Slice 23 do not come from the same approximate position.

So for each slice we calculate a relative position.

If the study has NN slices:

pi=iN−1p_i=\frac{i}{N-1}

This converts positions roughly into:

```
0 ————————— 0.5 ————————— 1
start       middle          end
```

For a 21-slice patient:

```
middle ≈ 0.5
```

For a 25-slice patient:

```
middle ≈ 0.5
```

So it works even when patients have different slice counts.

Then the attention mechanism gets:

```
DINOv2 feature
+
relative slice position
```

and learns:

what does the slice contain AND where approximately is it located within this MRI stack?

---

25. Why is position-aware attention useful?

Without position:

```
DINOv2 feature → what does this image look like?
```

With position:

```
DINOv2 feature → what does it look like?
Position       → where in the stack was it?
```

So the MRI is no longer treated as just an unordered collection of pictures.

---

26. What happens if patients have different slice counts?

This is completely manageable.

Patient A:

```
25 slices
→ 25 DINO vectors
→ attention
→ one 768-D patient vector
```

Patient B:

```
21 slices
→ 21 DINO vectors
→ attention
→ one 768-D patient vector
```

Patient C:

```
29 slices
→ 29 DINO vectors
→ attention
→ one 768-D patient vector
```

The output size is always fixed.

During batch training, we can pad shorter sequences:

```
Patient A = 25 real slices
Patient B = 21 real + 4 padded
```

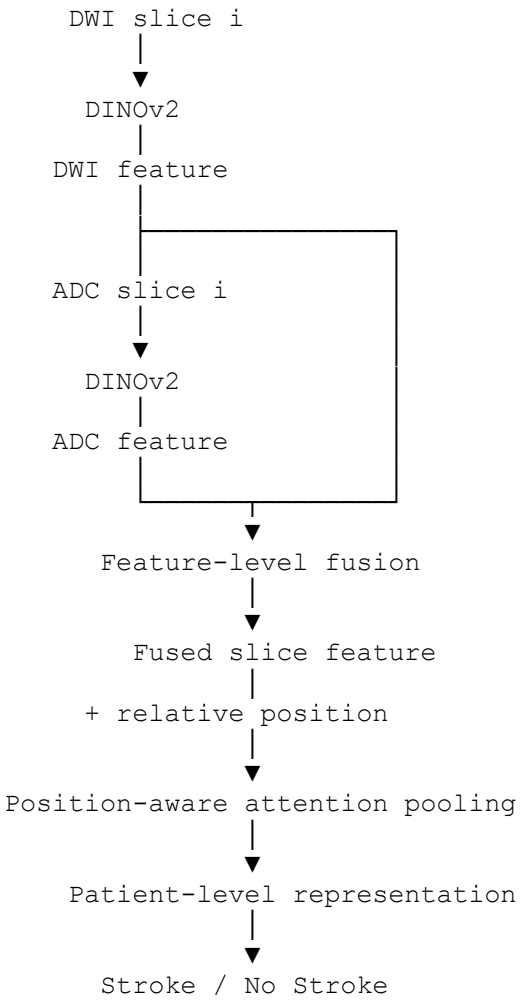
and provide a mask telling attention:

```
Ignore padded positions.
```

---

27. Module 2 architecture

Stroke module:



---

28. How exactly do DWI and ADC get fused?

Suppose DINOv2-Base produces:

DWIi→768−DDWI_i \rightarrow 768-D

and:

ADCi→768−DADC_i \rightarrow 768-D

We concatenate them:

[768+768]=1536−D[768 + 768] = 1536\text{-D}

Then use a small MLP:

1536→7681536 \rightarrow 768

So:

```
DWI feature ─┐
             ├→ concatenate → MLP → fused feature
ADC feature ─┘
```

This is called:

Feature-level fusion

It lets both sequences remain distinct until meaningful representations have been extracted.

---

29. Why not just make DWI and ADC RGB channels?

Because DWI and ADC are not simply different colors.

They are **different MRI measurements with different physical intensity meanings**.

So we don't want:

```
R = DWI
G = ADC
B = something
```

as the main approach.

Instead:

```
DWI → feature extraction
ADC → feature extraction
then fuse features
```

which is methodologically cleaner.

---

30. Major research gaps we are addressing

The literature shows that many pieces already exist independently.

But there are several gaps.

Gap: slice-level classification

Some papers classify individual MRI slices.

Our response:
patient-level attention pooling across the entire relevant MRI study.

Gap: manually selected abnormal slices

Some stroke work gives the model only obvious lesion-containing slices.

Our response:
analyse the full relevant study and let attention identify useful slices.

Gap: majority voting / simple averaging

These approaches either:

- give every slice equal weight
- or count binary predictions.

Our response:
learn the contribution of every slice using attention.

Gap: lack of explicit slice position

2D slice encoders do not inherently know where each slice occurred in the volume.

Our response:
relative positional information + attention.

Gap: fixed-volume CNN input

Some 3D CNN papers convert every MRI to a fixed number of slices such as 16.

This can require:

- interpolation
- downsampling
- discarding information.

Our response:
variable-length slice processing with DINOv2 + attention pooling.

Gap: limited DWI–ADC integration

Some stroke systems use DWI only.

Others use DWI/ADC but still perform simple slice-level classification.

Our response:
explicit paired DWI–ADC feature fusion before patient-level aggregation.

Gap: conventional CNNs dominate

Many approaches train CNNs/3D CNNs from scratch or heavily fine-tune them.

Our response:
explore pretrained self-supervised DINOv2 representations, especially useful for a smaller hospital dataset.

Gap: tasks are usually isolated

Literature often does:

```
abnormality classification only
OR
stroke only
OR
tumor classification only
```

Our response:

```
General abnormality branch
+
dedicated acute-stroke branch
+
clinical triage fusion
```

---

31. But what is actually novel?

Be careful with this.

Do **not** tell your friends:

"Nobody has ever used DINOv2 with MRI attention."

That isn't true.

Research already exists on DINOv2 + MRI slice aggregation.

A safer claim is:

The reviewed literature does not commonly combine **DINOv2-based patient-level volumetric aggregation, explicit relative slice position, paired DWI–ADC feature fusion, separate FLAIR abnormality classification and final clinical triage within one framework.**

We are integrating these ideas into one system.

---

32. Final decision fusion

A simple clinically understandable rule is:

```
IF Stroke = Yes
        ↓
    IMMEDIATE

ELSE IF FLAIR = Abnormal
        ↓
      URGENT

ELSE
        ↓
     ROUTINE
```

Possible additional rule:

```
Poor image quality / very uncertain prediction
                ↓
           NEEDS REVIEW
```

You can later refine this with your radiologist.

---

33. Explainability

We want two levels.

Slice-level explanation

Attention weights can show:

```
Slice 3   = 2%
Slice 4   = 4%
Slice 12  = 41%
Slice 13  = 35%
...
```

So the radiologist can see which slices mattered most.

Within-slice explanation

A ViT-compatible attribution method can highlight regions inside the chosen slice.

So:

```
Which slices mattered?
        ↓
attention pooling

Which regions inside the slice mattered?
        ↓
heatmap / attribution
```

Important:

A heatmap is not a lesion segmentation mask.

---

34. What preprocessing are we likely to perform?

For Module 1, roughly:

```
DICOM
→ QC
→ correct FLAIR series
→ brain localization/crop with safety margin
→ remove slices with almost no brain
→ intensity clipping/normalization
→ resize/pad
→ grayscale → 3 identical channels
→ DINOv2
```

For Module 2:

```
DICOM
→ identify DWI and ADC
→ geometry QC
```

```
→ pair/alignment
→ shared crop
→ sequence-appropriate normalization
→ resize
→ grayscale → 3 channels
→ DINOv2
```

Do not artificially colorize the MRI.

---

35. Why grayscale becomes 3 channels

DINOv2 was pretrained expecting three channels.

MRI has one grayscale intensity channel.

We can simply do:

```
MRI = X

Channel 1 = X
Channel 2 = X
Channel 3 = X
```

So it looks like:

[X,X,X][X,X,X]

This satisfies the input shape without inventing fake colors.

---

36. Training strategy

A sensible first experiment:

```
DINOv2 frozen
      |
      ▼
Train:
attention
fusion
classifier
```

Then compare against:

```
last few DINO blocks unfrozen
```

Ablation experiments should include things like:

```
DINOv2 + mean pooling
vs
DINOv2 + position-aware attention

DWI only
vs
DWI + ADC

Frozen DINOv2
vs
partial fine-tuning

CNN baseline
vs
DINOv2 framework
```

This tells us which part actually improves performance.

---

37. Metrics

Don't judge the model using only accuracy.

For Module 1:

```
Sensitivity
Specificity
Precision
F1
AUROC
Confusion matrix
```

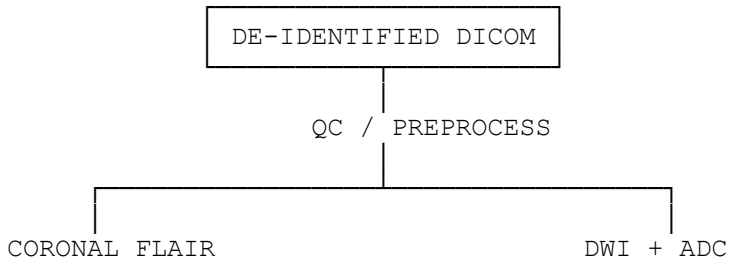
For stroke, sensitivity is especially important because a false-negative acute stroke is serious.

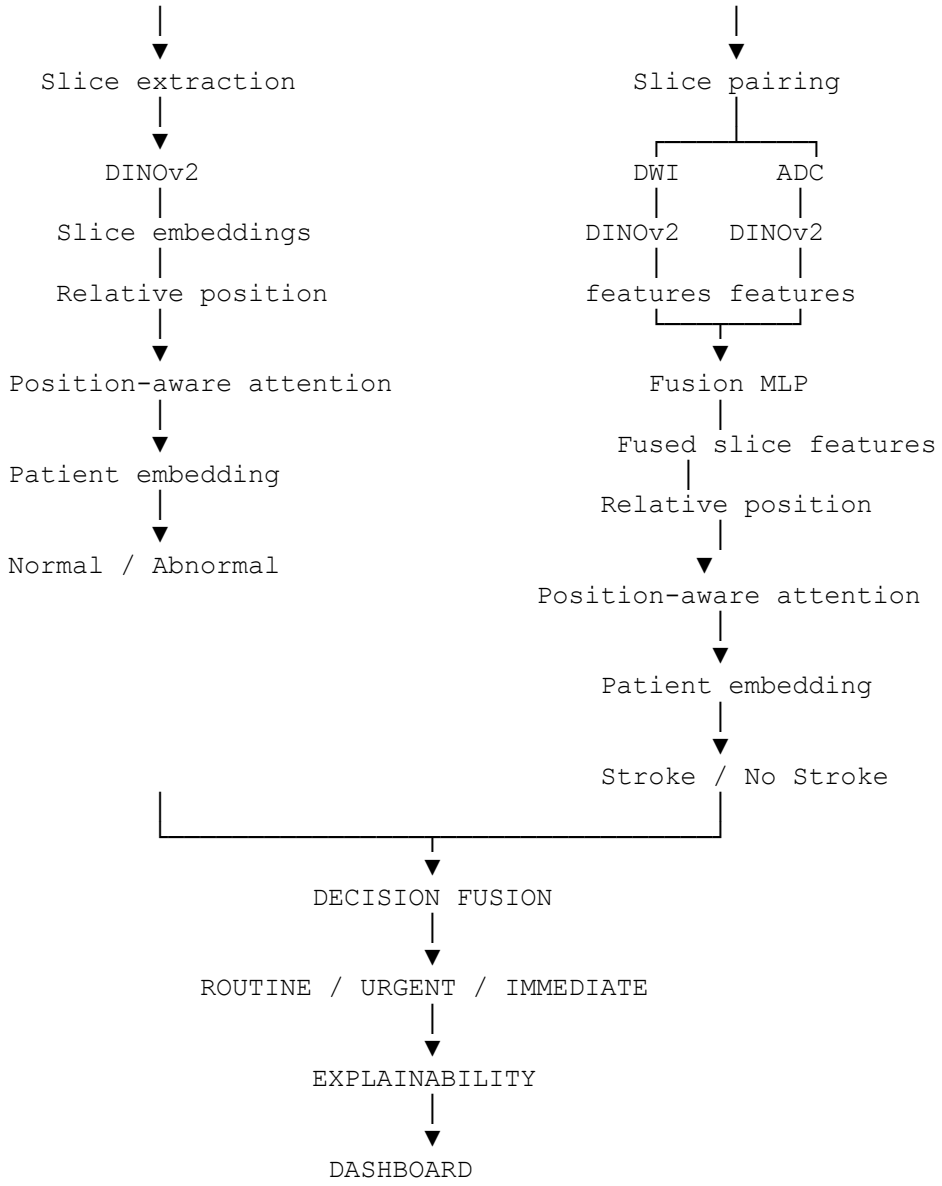
Also evaluate final triage:

```
under-triage
over-triage
urgent/immediate sensitivity
stroke false negatives
```

---

38. The complete project in one diagram





39. The simplest way to explain the whole idea to your teammates

You can tell them:

“We are building two patient-level MRI classifiers. One analyzes coronal FLAIR to determine whether the examination is generally normal or abnormal, and the other analyzes paired DWI and ADC to detect acute ischemic stroke. DINOv2 is used to extract a feature representation from every individual MRI slice. Because each patient can have a different number of slices, we do not force all studies into a fixed volume. Instead, we attach the relative position of each slice and use position-aware attention pooling to learn which slices are most important and combine them into one patient-level representation. For the stroke module, DWI and ADC features from corresponding slices are fused before the attention stage. The outputs of both modules are then combined to produce Routine, Urgent or Immediate triage priority.”

Question: i just have a doubt regarding our project.. so each patient may have different number of slices like one may have 25, other may have 21 and so on... how do we handle that or is dinov2 capable of handling that ?
ANSWER:

Yes — this is actually one of the reasons our proposed **DINOv2 + attention-pooling approach fits your project well**.

The important distinction is:

DINOv2 itself does not process the whole 21-slice or 25-slice MRI study at once. It processes each 2D slice individually.

So Patient A having 25 slices and Patient B having 21 slices is completely manageable.

Example

For **Patient A with 25 FLAIR slices**:

25 slices → DINOv2 applied 25 times → 25 feature vectors

For **Patient B with 21 slices**:

21 slices → DINOv2 applied 21 times → 21 feature vectors

If we're using DINOv2-B and a 768-D representation:

PatientA→25×768Patient_A \rightarrow 25 \times 768 PatientB→21×768Patient_B \rightarrow 21 \times 768

DINOv2 does **not care that the patients have different numbers of slices**, because it only sees one resized 2D image at a time.

Then attention pooling handles the variable number of slices

For Patient A:

z1,z2,…,z25z_1,z_2,\ldots,z_{25}

For Patient B:

z1,z2,…,z21z_1,z_2,\ldots,z_{21}

The attention layer calculates weights over **whatever number of slices are available**.

For Patient A:

α1+α2+⋯+α25=1\alpha_1+\alpha_2+\cdots+\alpha_{25}=1

For Patient B:

α1+α2+⋯+α21=1\alpha_1+\alpha_2+\cdots+\alpha_{21}=1

Then:

zpatient=∑iαiziz_{patient}=\sum_i \alpha_i z_i

Regardless of whether there were **21, 25, or 30 slices**, the final output is always **one fixed-size patient embedding**, for example:

768−D768-D

That 768-D patient embedding can then go into the classifier.

What about position if patients have different slice counts?

This is exactly why we use **relative position**, not just absolute slice number.

For each patient:

pi=iN−1p_i=\frac{i}{N-1}

So if one patient has 21 slices and another has 25, their positions are still normalized roughly between:

0→10 \rightarrow 1

For example:

Location	21-slice patient	25-slice patient
Beginning	0	0
Middle	~0.5	~0.5
End	1	1

Therefore the model understands:

“this slice is approximately in the middle of this patient's stack”

rather than depending on something arbitrary like **slice number 11**.

So we do NOT need to force every patient to 16 or 25 slices

That's actually an advantage compared with a fixed-volume approach such as the Gauriau paper.

We don't need:

25 slices → artificially reduce to 16

or

21 slices → duplicate/interpolate until 25.

Instead:

Use the relevant available slices → DINOv2 → position-aware attention pooling.

We would still perform basic **slice filtering**, for example removing extreme slices with almost no brain tissue, but we don't need every patient to end up with exactly the same number.

One small technical detail during training

Neural networks usually train multiple patients in one batch, and tensors like equal dimensions.

Suppose one batch contains:

- Patient 1 → 21 slices
- Patient 2 → 25 slices
- Patient 3 → 23 slices

We can temporarily pad them to the maximum:

- Patient 1 → 21 real + **4 padded**
- Patient 2 → 25 real
- Patient 3 → 23 real + **2 padded**

Then we use an **attention mask** telling the network:

"Ignore these padded positions."

So padded slices receive no attention and do not influence the prediction.

Conceptually:

Variable number of real slices → padding only for batching → mask padded values → attention over real slices only.

So the complete idea is

Patient with N slices

→ preprocess each slice
→ DINOv2 on every slice
→ NN feature vectors
→ add relative position
→ **position-aware attention pooling over N slices**
→ one fixed-size patient representation
→ **Normal / Abnormal**

And the exact same principle works for Module 2, except the units being aggregated are **paired DWI–ADC slice features**.

So yes: **different slice counts are not a problem for our architecture. In fact, variable-length patient studies are one of the reasons we're choosing slice-level DINOv2 features + attention pooling rather than forcing everyone into a fixed 3D volume.**